

Informe del Proyecto Minix

Sistema Operativo

Facultad de Matemática y Computación

Universidad de La Habana

Integrante: Kamila Reinoso Asin C211

1. Introducción

El presente proyecto se desarrolla en el marco de la asignatura Sistemas Operativos de la Facultad de Matemática y Computación de la Universidad de La Habana, con el fin de profundizar en los pilares que sustentan la computación moderna. A lo largo del trabajo, se abordan principios fundamentales como la abstracción de procesos —entendidos como entidades activas con estado dinámico—, la concurrencia y sincronización de hilos, la planificación de ejecuciones para la gestión eficiente del tiempo de CPU y la interacción con el sistema de archivos. Se utiliza MINIX como plataforma de estudio por su diseño modular y pedagógico, el cual permite intervenir directamente en componentes reales del núcleo y las bibliotecas del sistema, facilitando la transición de los conceptos teóricos de la multiprogramación a la implementación práctica.

El objetivo general de este proyecto es consolidar el aprendizaje mediante la modificación y el análisis interno de un sistema operativo funcional, promoviendo la capacidad de diagnosticar fallos y proponer soluciones a nivel de sistema. Para ello, el desarrollo se estructura en cuatro hitos específicos: la personalización del entorno de usuario mediante el mensaje de bienvenida; la depuración de un error crítico en la biblioteca de hilos en el método `pthread_mutex_trylock` para garantizar la exclusión mutua; la implementación del comando `tree`, que requiere el uso de llamadas al sistema para la navegación recursiva de directorios; y, finalmente, la modificación del scheduler. En este último punto, se implementa una política de penalización para procesos intensivos en CPU, basada en ventanas temporales, con el fin de optimizar la respuesta del sistema ante tareas interactivas

2. Desarrollo y resultados por componente

2.1 Instalación y configuración de Virtual Box y Minix

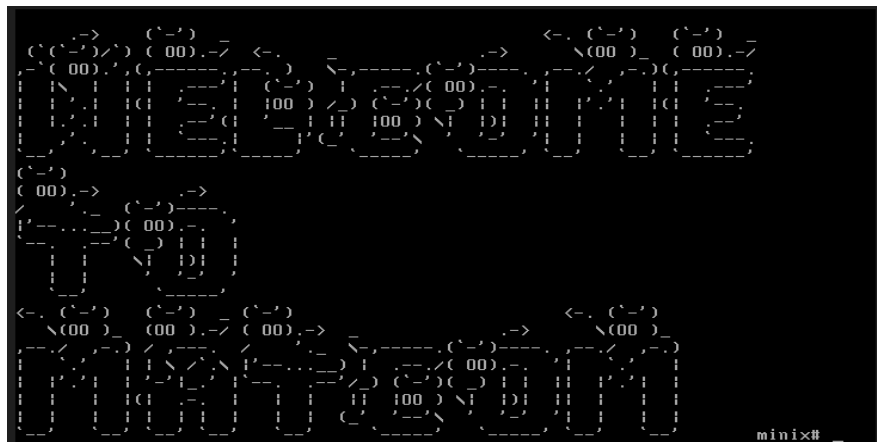
Para la preparación del entorno de trabajo, se configuró una máquina virtual con los siguientes parámetros técnicos y procedimientos de software:

- Software de virtualización: Oracle Virtual Box versión 7.2.6.
- Sistema operativo invitado: MINIX versión 3.4.
- Memoria RAM: 2 GB. Esta asignación garantiza la fluidez necesaria para la compilación del kernel y la ejecución de pruebas de concurrencia, superando el mínimo recomendado.
- Procesador: 3 núcleos de CPU, seleccionados para optimizar los tiempos de procesamiento durante el desarrollo de los hitos.

- ✚ Almacenamiento: 40 GB de disco duro virtual, permitiendo el manejo de múltiples binarios y logs de depuración.
- ✚ Configuración de red: Adaptador en modo NAT para facilitar la conexión a los repositorios oficiales.
- ✚ Instalación de herramientas de desarrollo:
 - Se utilizó el gestor de paquetes para instalar git-base, permitiendo la sincronización con el repositorio del proyecto.
 - Se empleó el comando `pkgin_sets` para visualizar e instalar los conjuntos de paquetes (*sets*) estándar requeridos para las tareas de desarrollo y compilación en el sistema.
- ✚ Estado de la instalación: El proceso se completó satisfactoriamente siguiendo la documentación oficial, logrando un entorno estable para la ejecución del proyecto.

2.2 Personalización del mensaje de bienvenida

Para poder editar el mensaje de bienvenida de MINIX en el archivo `motd` usamos la página ASCII ART GENERATOR, con el mensaje WELCOME TO MATCOM, copiado el mensaje de bienvenida en la dirección `etc/motd`, dado que el proyecto se ha desarrollado en el sistema operativo Ubuntu y probado su funcionamiento en Minix hemos subido este mensaje a github, descargado en Minix mediante el comando `git -c http.sslVerify=false pull origin Hito-1` dado que al hacer solo `git pull origin Hito-1` nos ha dado un error en el certificado SSL solucionado con el comando anterior. Se usa el comando `cp etc/motd /etc/motd` para copiar el archivo descargado desde GitHub en el sistema de minix a continuación se aporta una imagen de dicho mensaje ya puesto en el sistema.



2.3 Depuración del bug en pthread

El fallo en la biblioteca de hilos de MINIX es un error de diseño en la capa de compatibilidad que impide el funcionamiento correcto de la exclusión mutua cuando se utiliza el estándar POSIX.

- Síntomas del fallo

El problema se manifiesta cuando un programa intenta realizar llamadas sucesivas a la función `pthread_mutex_trylock` sobre un mismo mutex. En un comportamiento normal, la primera llamada debería bloquear el mutex y la segunda debería retornar inmediatamente con un código de error (como `EDEADLK` o `EBUSY`) indicando que el mutex ya está ocupado. Sin embargo, en la versión con el bug, el programa se bloquea indefinidamente en la segunda

llamada, quedando "colgado" antes de poder ejecutar las operaciones de desbloqueo (unlock) o destrucción (destroy).

- Análisis de la causa raíz

La arquitectura de hilos en MINIX divide las funciones en dos niveles:

Capa nativa (mthread_*): Implementada en libmthread, es la lógica real que gestiona los mutexes en el sistema.

Capa de compatibilidad (pthread_*): Se encuentra en el archivo minix/lib/libmthread/pthread_compat.c y su función es simplemente "traducir" las llamadas de estándar POSIX a las funciones nativas de MINIX.

El error específico: Dentro de pthread_compat.c, la función pthread_mutex_trylock contenía una recursión infinita. En lugar de invocar a la función nativa del sistema (mthread_mutex_try_lock), el código se llamaba a sí mismo repetidamente. Esta sucesión de llamadas circulares agota el espacio de la pila (stack overflow), provocando que el proceso se bloquee y no pueda retornar nunca el control al programa de usuario.

- Corrección técnica

La solución consiste en romper la recursión redirigiendo la llamada a la función correcta de la capa nativa. En el archivo pthread_compat.c, se modificó el retorno de la siguiente manera:

```
--- a/libmthread/pthread_compat.c
+++ b/libmthread/pthread_compat.c
- return pthread_mutex_try_lock(mutex);
+ return mthread_mutex_try_lock(mutex);
```

- Verificación de la solución

Una vez aplicada la corrección, el programa de prueba arrojó los siguientes resultados:



```
minix# cd /usr/src/minix/tests
minix# ls -F
Makefile      dev/          ipf/          net/          test_hito2_bug/
Makefile.inc  fs/           kernel/       rump/         test_hito3/
README        games/        lib/          sbin/         usr.bin/
bin/          h_macros.h   libexec/     share/        usr.sbin/
crypto/       include/     modules/     sys/
minix# cd test_hito2_bug
minix# ls -F
program_test*  test_bug.c
minix# ./program_test
first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
minix#
```

2.4 Implementación del comando tree

En esta sección se documenta el desarrollo y la integración del comando tree, una herramienta diseñada para visualizar de manera jerárquica y recursiva la estructura de directorios y archivos en el sistema MINIX.

Diseño del algoritmo

Para la implementación de este comando, se optó por un enfoque de recursión explícita mediante la función `tree (char *dir, int level)`, descartando una solución iterativa basada en una pila manual. Esta decisión se fundamentó en tres pilares técnicos extraídos del análisis del problema:

- ✚ Afinidad con la estructura de datos: Dado que los sistemas de archivos son estructuras jerárquicas o arbóreas por definición, la recursión permite modelar su recorrido de forma natural y directa, tratando cada subdirectorio como la raíz de un nuevo subárbol.
- ✚ Simplicidad en la indentación: El manejo de la profundidad visual mediante el parámetro `level` es significativamente más sencillo. Al pasar este valor en cada llamada, el programa determina con precisión el número de espacios de indentación sin necesidad de gestionar variables de estado globales o complejas.
- ✚ Legibilidad y mantenimiento: Al delegar la gestión del retroceso (*backtracking*) a la pila de llamadas del sistema operativo, se evita la implementación manual de estructuras de datos adicionales (como un *stack*), lo que resulta en un código más compacto, legible y menos propenso a errores de gestión de memoria.

Llamadas al sistema utilizadas

Para interactuar con la capa del sistema de archivos de MINIX, se emplearon las siguientes llamadas al sistema estándar de C/POSIX:

- ✚ `opendir`: Permite abrir el flujo del directorio especificado en la ruta para iniciar su lectura.
- ✚ `readdir`: Se utiliza para leer secuencialmente cada entrada (archivo, directorio o enlace) contenida dentro del flujo abierto.
- ✚ `lstat`: Fundamental para obtener la información de estado de la entrada. Se seleccionó específicamente sobre `stat` para obtener los datos del enlace simbólico mismo y no del objeto al que apunta, cumpliendo con el requisito de seguridad del proyecto.
- ✚ `closedir`: Cierra el flujo del directorio para liberar los descriptores de archivos y recursos del sistema una vez finalizado el procesamiento.

Indentación y visualización de profundidad

La representación visual de la jerarquía se gestiona a través del argumento `level`. Por cada nivel de profundidad alcanzado en la recursión, el programa ejecuta un ciclo que imprime una cantidad proporcional de espacios en blanco (o caracteres visuales) antes del nombre de la entrada. Esto garantiza que los archivos y carpetas hijos aparezcan alineados a la derecha respecto a su directorio padre, facilitando la lectura de estructuras complejas.

Prevención de ciclos y enlaces simbólicos

Un punto crítico del diseño es la prevención de bucles infinitos causados por enlaces simbólicos. Siguiendo las restricciones del proyecto, el programa utiliza `lstat` en conjunto con la macro `S_ISLNK` o verificaciones de tipo de archivo para identificar enlaces. El algoritmo está configurado para no seguir enlaces simbólicos de forma **recursiva**, asegurando que solo se exploren directorios reales. Además, se implementó un filtrado estricto para ignorar las entradas. (directorio actual) y `..` (directorio padre), evitando que la recursión regrese a niveles superiores.

Lógica principal y manejo de argumentos

- ✚ Punto de entrada (main): El programa acepta una ruta como argumento desde la línea de comandos (argv). En caso de no proporcionarse ninguna, el sistema asume por defecto el directorio actual (.).
- ✚ Construcción de rutas dinámicas: Se utiliza la función snprintf para concatenar la ruta base con el nombre de la entrada actual, generando una ruta completa válida que se pasa como argumento a la siguiente llamada recursiva.

Integración en el sistema

El código fuente se alojó en el directorio oficial de comandos del sistema: /usr/src/minix/commands/tree. Para su correcta compilación e integración definitiva en los binarios de MINIX, se creó un Makefile con la siguiente configuración técnica:

- ✚ PROG = tree: Especifica el nombre del binario a generar.
- ✚ SRCS = tree.c: Define el archivo de código fuente principal.
- ✚ .include <bsd.prog.mk>: Incorpora las macros e instrucciones estándar de compilación del sistema BSD/MINIX, permitiendo que el comando sea instalado y reconocido globalmente por el sistema operativo

Ejemplo con comando tree /usr

```
minix# tree /usr
```

```

gtest-typed-test.cc
gtest.cc
update_llc_test_checks.py
valgrind
i386-pc-linux-gnu.supp
x86_64-pc-linux-gnu.supp
vim
  README
  llvm.vim
  taglegen.vim
  vimrc
wc11a.py
yaml-bench
CMakeLists.txt
Makefile
YAMLBench.cpp
include
  Makefile
lib
  Makefile
  Makefile.inc
  libLLVMArch64AsmParser
  Makefile
  libLLVMArch64AsmPrinter
```

Ejemplo con la ruta /usr/src/minix/minix/servers/sched

```
minix# tree /usr/src/minix/minix/servers/sched
Makefile
main.c
proto.h
sched.h
schedproc.h
schedule.c
utility.c
gdinit
main.o
schedule.o
utility.o
sched
minix# _
```

Ejemplo con la ruta actual

```
minix# cd /usr/src/minix/tests
minix# tree
```

```

d_merge_C.F.S.out
mtree_d_create.out
netbsd6_d_create.out
t_mtree.sh
tcpdump
  Makefile
  t_tcpdump.sh
traceroute
  Makefile
  t_traceroute.sh
useradd
  Makefile
  t_useradd.sh
test_hito2_bug
test_bug.c
program_test
test_hito3
CPU_bound.c
Intensivo.c
Mixto.c
a.out
cpu_bound
intensivo
mixto
minix#
```

- ✚ Comparación con el comando tree estándar

La implementación realizada en este proyecto guarda una estrecha paridad funcional con el comando `tree` estándar de sistemas tipo Unix/Linux, cumpliendo con los objetivos de visualización y navegación jerárquica. A continuación, se detallan las similitudes y las decisiones técnicas adoptadas:

- ✚ Comportamiento y argumentos: Al igual que la herramienta estándar, nuestro comando acepta una ruta opcional como argumento y, en su ausencia, asume por defecto el directorio actual (`.`). El flujo de ejecución es idéntico, permitiendo al usuario explorar tanto rutas relativas como absolutas de manera intuitiva.
- ✚ Representación visual: Se ha replicado la jerarquía escalonada mediante una lógica de indentación basada en la profundidad del árbol. Aunque el comando estándar suele utilizar caracteres gráficos (como `|—` o `└—`), nuestra versión utiliza espacios proporcionales al nivel de recursión, lo cual garantiza una visualización limpia y compatible con la terminal básica de MINIX sin necesidad de bibliotecas de caracteres especiales.
- ✚ Gestión de archivos y directorios: El algoritmo procesa y muestra de forma indistinta tanto archivos como subdirectorios, manteniendo la recursividad completa hasta agotar las ramas del sistema de archivos, tal como lo hace la herramienta de referencia.
- ✚ Seguridad y Enlaces Simbólicos: Un punto crítico de coincidencia es el manejo de la estabilidad. Al igual que el `tree` de Linux en su modo predeterminado, nuestra implementación utiliza `lstat` para detectar enlaces simbólicos y evitar seguir sus rutas de forma recursiva. Esto previene ciclos infinitos, una característica de robustez esencial en cualquier herramienta de navegación de sistemas de archivos.
- ✚ Integración nativa: A diferencia de una herramienta externa, nuestro comando ha sido integrado directamente en el sistema de construcción de MINIX mediante un `Makefile` basado en `bsd.prog.mk`, lo que lo convierte en una instrucción nativa disponible globalmente en la shell, operando con la misma eficiencia que los comandos integrados del sistema operativo.

2.5 Penalización por uso intensivo de la CPU

Esta sección detalla la modificación más profunda realizada al núcleo de MINIX: la transición de un esquema de planificación estático a uno dinámico basado en el historial de consumo de CPU. El objetivo es diferenciar y priorizar procesos interactivos frente a aquellos con una carga de cómputo ininterrumpida (*CPU-bound*).

2.5.1. Análisis teórico de la planificación en MINIX

Para comprender las modificaciones, es imperativo analizar los dos algoritmos que rigen el comportamiento nativo del sistema:

- ✚ **MLFQ (Multi-level Feedback Queue):** MINIX organiza los procesos listos para ejecutar en múltiples colas de prioridad. El planificador siempre selecciona el proceso en la cola de mayor prioridad disponible. Si un proceso agota su tiempo asignado, es desplazado a una cola de menor prioridad.
- ✚ **RR (Round Robin):** Cuando existen múltiples procesos dentro de una misma cola (misma prioridad), el sistema aplica un esquema de Round Robin. A cada tarea se le asigna un intervalo de tiempo fijo (*quantum*); una vez agotado, el

proceso cede el turno al siguiente en la cola de forma cíclica para garantizar equidad

2.5.2. Análisis previo y estado inicial del Scheduler

En el diseño original de MINIX, antes de nuestra intervención, la política de planificación presentaba una rigidez técnica significativa que afectaba la respuesta del sistema ante procesos mixtos:

- ✚ **Gestión del Quantum (do_noquantum):** Localizado en /usr/src/minix/servers/sched/schedule.c, este mensaje se activa cuando un proceso agota su tiempo de CPU. Originalmente, el sistema reducía la prioridad del proceso de forma inmediata y sin considerar su comportamiento previo, lo que penalizaba por igual ráfagas cortas de actividad y consumos sostenidos

```
83  /*-----*/
84  *          do_noquantum          *
85  *-----*/
86
87  int do_noquantum(message *m_ptr)
88  {
89      register struct schedproc *rmp;
90      int rv, proc_nr_n;
91
92      if (sched_isokendpt(m_ptr->m_source, &proc_nr_n) != OK) {
93          printf("SCHED: WARNING: got an invalid endpoint in OQ msg %u.\n",
94                m_ptr->m_source);
95          return EBADEPT;
96      }
97
98      rmp = &schedproc[proc_nr_n];
99      if (rmp->priority < MIN_USER_Q) {
100          rmp->priority += 1; /* lower priority */
101      }
102
103      if ((rv = schedule_process_local(rmp)) != OK) {
104          return rv;
105      }
106      return OK;
107  }
```

- ✚ **Mecanismo de Balanceo (balance_queues):** Para evitar la inanición (*starvation*) de los procesos en colas bajas, la función `balance_queues` incrementaba la prioridad de todos los procesos de usuario en una unidad cada 5 segundos. Esta acción era indiscriminada y no evaluaba si el proceso realmente merecía recuperar prioridad por haber estado bloqueado o haber cedido la CPU voluntariamente.

```
353  void balance_queues(void)
354  {
355      struct schedproc *rmp;
356      int r, proc_nr;
357
358      for (proc_nr=0, rmp=schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++) {
359          if (rmp->flags & IN_USE) {
360              if (rmp->priority > rmp->max_priority) {
361                  rmp->priority -= 1; /* increase priority */
362                  schedule_process_local(rmp);
363              }
364          }
365      }
366
367      if ((r = sys_setalarm(balance_timeout, 0)) != OK)
368          panic("sys_setalarm failed: %d", r);
369  }
```

- ✚ **Estructuras de Datos:** La información del proceso se gestiona en la estructura `schedproc` (definida en `schedproc.h`). En el estado inicial, esta estructura carecía

de contadores para rastrear el uso acumulado de CPU dentro de ventanas temporales específicas, limitándose a almacenar la prioridad actual (*priority*) y la máxima permitida (*max_priority*)

2.5.3. Diseño de la política modificada

Tras el análisis del planificador original, se diseñó e implementó una política dinámica que ajusta las prioridades de los procesos basándose en su historial de consumo real de tiempo de CPU. El diseño se fundamenta en los siguientes parámetros y decisiones técnicas:

- ✚ **Límite de penalización (N = 5):** Se estableció la constante **LIMIT en 5**. Esto define que un proceso solo será penalizado con una bajada de prioridad si agota su *quantum* de tiempo cinco veces dentro de una misma ventana de observación. Este valor permite al sistema diferenciar con mayor precisión entre ráfagas cortas de actividad y procesos genuinamente intensivos en CPU (*CPU-bound*).

```
#define BALANCE_TIMEOUT 5 /* how often to balance queues in seconds */
#define LIMIT 5 /* number of quanta a process can consume before we lower its priority */
static unsigned balance = 0;
static int schedule_process(struct schedproc *rmp, unsigned flags);
```

- ✚ **Ventana de Priority Boost (25 segundos):** Se implementó un mecanismo de "Gran Salto" que se activa cada **25 segundos** (equivalente a 5 ciclos de balanceo de 5 segundos cada uno). En este intervalo, el sistema restaura la prioridad de todos los procesos de usuario a su valor máximo permitido (*max_priority*), mitigando de forma efectiva el riesgo de inanición (*starvation*).
- ✚ **Optimización del rendimiento del kernel:** Se rediseñó la lógica de actualización para que la función *schedule_process_local* solo sea invocada cuando existe un cambio efectivo en la prioridad del proceso. Esto evita que el kernel consuma ciclos de CPU innecesarios refrescando procesos que no han sufrido modificaciones en su estado.

```
92 void balance_queues(void)
93 {
94     balance += 1;
95     struct schedproc *rmp;
96     int r, proc_nr;
97     // Subir la prioridad solo a los que no han gastado ningun quantum
98     for (proc_nr = 0, rmp = schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++)
99     {
100         if (rmp->flags & IN_USE)
101         {
102             if (rmp->priority > rmp->max_priority)
103             {
104                 //printf("SCHED: balanceando pid=%d priority=%d\n, max=%d count=%d\n", proc_nr, rmp->priority,
105                 // balanceando pid=%d priority=%d\n, max=%d count=%d\n", proc_nr, rmp->priority,
106                 // rmp->max_priority, rmp->count_quantums);
107                 if (balance >= 5)
108                 {
109                     //printf("=====Entre al priority boost =====\n");
110                     //printf("SCHED: balanceando pid=%d priority=%d\n, max=%d count=%d\n", proc_nr, rmp->priority,
111                     // rmp->max_priority, rmp->count_quantums);
112                     rmp->priority = rmp->max_priority; /* increase priority */
113                     schedule_process_local(rmp);
114                 }
115                 else if (rmp->count_time_slices == 0)
116                 {
117                     //printf("=====Balanceo normal===== \n");
118                     //printf("SCHED: balanceando pid=%d priority=%d\n, max=%d count=%d\n", proc_nr, rmp->priority,
119                     // rmp->max_priority, rmp->count_quantums);
120                     rmp->priority = 1; /* increase priority */
121                     schedule_process_local(rmp);
122                 }
123             }
124             rmp->count_time_slices = 0;
125             rmp->count_quantums = 0; /* reset quantum count */
126         }
127     }
128     if (balance >= 5)
129     {
130         balance = 0;
131     }
132     if ((r = sys_setalarm( BALANCE_TIMEOUT * sys_hz(), 0)) != OK)
133     {
134         panic("sys_setalarm failed: %d", r);
135     }
136 }
```


-  **Protección contra penalizaciones contradictorias:** Se introdujo una lógica de control para que el sistema ignore las notificaciones de agotamiento de *quantum* durante la ejecución del *Priority Boost*, asegurando que un proceso recién beneficiado por la recuperación global no sea penalizado inmediatamente de forma injusta.

2.5.4. Cambios realizados y Reconstrucción del Sistema

Para integrar estas modificaciones, se alteraron los archivos fundamentales del servicio sched. A continuación, se detallan los cambios línea por línea:

- A. Modificaciones en schedproc.h Se añadieron campos específicos a la estructura schedproc para permitir el rastreo del comportamiento de los procesos:

- unsigned count_quantums: Contador de *quantums* completos agotados por el proceso.
- unsigned count_time_slices: Contador de ventanas temporales para el control del ciclo de 25 segundos.

```

23 EXTERN struct schedproc
24 {
25     endpoint_t endpoint; /* id de endpoint del proceso */
26     endpoint_t parent; /* id de endpoint del padre */
27     unsigned flags; /* bits de banderas */
28
29     /* Planificación en espacio de usuario */
30     unsigned max_priority; /* la prioridad máxima permitida de este proceso */
31     unsigned priority; /* la prioridad actual del proceso */
32     unsigned time_slice; /* el quantum de tiempo de este proceso */
33     unsigned count_quantums; /* el número de quantums que el proceso ha consumido por ventana */
34     unsigned count_time_slices; /* en que CPU se está ejecutando el proceso */
35     unsigned cpu; /* en que CPUs tiene permitido ejecutarse el proceso */
36     bitmap_t cpu_mask[BITMAP_CHUNKS(CONFIG_MAX_CPUS)];
37 } schedproc[NR_PROCS];
38
39 /* Flag values */
40 #define IN_USE 0x0001 /* set when 'schedproc' slot in use */

```

- B. Modificaciones en schedule.c - Función do_noquantum Esta función, encargada de gestionar el agotamiento del tiempo asignado, fue modificada con la siguiente lógica:

- **Validación de Boost:** Se incluyó un bloque condicional if que verifica si el sistema se encuentra en medio de un rebalanceo global. Si el sistema está "balanceando", la función retorna OK inmediatamente, protegiendo al proceso de penalizaciones durante el *boost*.
- **Contabilización y Evaluación:** Se incrementa el contador rmp->count_quantums++. Solo si este valor alcanza el LIMIT (5) y la prioridad actual es superior a la mínima (MIN_USER_Q), se procede a bajar la prioridad (rmp->priority += 1).

```

82 do_noquantum
83 {
84     int do_noquantum(message *m_ptr)
85     {
86         register struct schedproc *rmp;
87         int rv, proc_nr_n;
88
89         if (sched_isokendpt(m_ptr->source, &proc_nr_n) != OK)
90             return EBADEPT;
91
92         rmp = &schedproc[proc_nr_n];
93         /* Contador de quantums gastados por el proceso */
94         rmp->count_quantums++;
95         rmp->count_time_slices++;
96         if (balance == 0)
97         {
98             rmp->count_quantums++;
99             rmp->count_time_slices++;
100             rmp->count_time_slices++;
101             /* Si el proceso se ha quedado sin quantum demasiadas veces, bajamos su prioridad */
102             if (rmp->priority < MIN_USER_Q)
103             {
104                 if (rmp->count_quantums == LIMIT)
105                 {
106                     //printf("=====Estoy penalizando=====n");
107                     //printf("El proceso %d/n", rmp->id);
108                     rmp->priority += 1; /* lower priority */
109                     rmp->count_quantums = 0; /* reset quantum count */
110                 }
111             }
112         }
113
114         if ((rv = schedule_process_local(rmp)) != OK)
115             return rv;
116
117         return OK;
118     }
119 }

```

- C. Modificaciones en schedule.c - Función balance_queues Se ajustó el mecanismo de temporización para manejar tanto la recuperación gradual como el boost masivo:

- **Control del ciclo de 25s:** Se utiliza un contador global (balance) para identificar el momento en que se han cumplido 5 ciclos de 5 segundos.

- **Lógica de Boost:** Al alcanzar el ciclo de 25 segundos, todos los procesos recuperan su max_priority de forma inmediata y se resetean sus contadores de *quantums*.
- **Recuperación condicionada:** En los ciclos normales, solo aquellos procesos que no agotaron ningún *quantum* en la ventana anterior (count_time_slices == 0) son elegibles para subir un nivel de prioridad (rmp->priority -= 1).
- **Eficiencia:** Optimización de llamadas locales, reduciendo la carga del sistema durante el balanceo de colas.

```

392 void balance_queues(void)
393 {
394     balance -= 1;
395     struct schedproc *rmp;
396     int r, proc_nr;
397     // Subir la prioridad solo a los que no han gastado ningun quantum
398     for (proc_nr = 0, rmp = schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++)
399     {
400         if (rmp->flags & IN_USE)
401         {
402             if (rmp->priority > rmp->max_priority)
403             {
404                 //printk("SCHED: balanceando pid=%d priority=%d\n", max=hd count=hd\n", proc_nr, rmp->priority);
405                 // rmp->max priority, rmp->count.quantums);
406                 if (balance == 5)
407                 {
408                     //printk("====Entre al priority boost =====\n");
409                     //printk("SCHED: balanceando pid=%d priority=%d\n", max=hd count=hd\n", proc_nr, rmp->priority);
410                     // rmp->max priority, rmp->count.quantums);
411                     rmp->priority = rmp->max_priority; /* increase priority */
412                     schedule_process_local(rmp);
413                 }
414                 else if (rmp->count.time_slices == 0)
415                 {
416                     //printk("=====Balance normal===== \n");
417                     //printk("SCHED: balanceando pid=%d priority=%d\n", max=hd count=hd\n", proc_nr, rmp->priority);
418                     // rmp->max priority, rmp->count.quantums);
419                     rmp->priority -= 1; /* increase priority */
420                     schedule_process_local(rmp);
421                 }
422                 rmp->count.time_slices = 0;
423                 rmp->count.quantums = 0; /* reset quantum count */
424             }
425             if (balance == 5)
426             {
427                 balance = 0;
428             }
429             if (if (if = sys_setalarm(BALANCE_TIMEOUT * sys_hz(), 0)) != OK)
430                 panic("sys_setalarm failed: %d", r);
431         }
432     }
433 }

```

D. **Recompilación del núcleo** Para que los cambios surtan efecto en el sistema operativo, se ejecutó una reconstrucción integral:

- Limpieza e instalación del servicio: make clean && make && make install dentro de /usr/src/minix/servers/sched.
- Compilación global: make build en /usr/src para integrar la nueva lógica en el binario del kernel y reiniciar el sistema con la política activa.

E. **Validación experimental** Se crearon los procesos de cpu_bound, intensivo y mixto para verificar la validez del algoritmo del scheduler. El proceso de cpu_bound contiene un ciclo infinito que no realiza ninguna función solo consumir CPU, el proceso de intensivo es un ciclo infinito que se bloquea por cada iteración, y el proceso mixto es una mezcla entre ambos procesos. En las imágenes se puede observar los 3 procesos ejecutándose al mismo tiempo. En el caso de no ver el proceso de cpu_bound en 7 es que dado un límite tan bajo este proceso es capaz de gastar esta cantidad de quantums en unidades muy pequeñas de tiempo.

```

load averages: 0.57, 0.10, 0.03 up 0:01:46:55 00:30:30
72 processes: 2 runnable, 45 sleeping
CPU states: 100% user, 0.0% nice, 0.0% system, 0.0% kernel, 0.0% idle
Memory: 2047M Total, 2011M Free, 1996M Contig, 16M Cached

PID USERNAME PRI NICE SIZE RES STATE TIME WCPU CPU COMMAND
702 root 15 0 1224K 260K RUN 0:21 90.43% 62.01% cpu_bound
703 root 7 0 1240K 352K select 0:02 5.70% 3.56% mixto
0 root 0 0 3264K 3264K kernel 0:00 0.68% 0.68% (kernel)
33 service 7 0 204K 204K (amp) 0:02 0.00% 0.00% at_wini
256 root 7 0 160K 160K (amp) 0:01 0.00% 0.00% lance
10 root 7 0 432K 432K RUN 0:00 0.00% 0.00% mb
265 service 7 0 1304K 1304K (amp) 0:00 0.00% 0.00% luip
36 root 7 0 1168K 1168K (amp) 0:00 0.00% 0.00% ls
555 root 7 0 2232K 840K select 0:00 0.00% 0.00% syslogd
475 root 7 0 944K 640K select 0:00 0.00% 0.00% dhcpd
650 root 7 0 824K 572K wait 0:00 0.00% 0.00% sh
41 root 7 0 564K 560K (amp) 0:00 0.00% 0.00% procs
705 root 7 0 736K 508K sysctl 0:00 0.00% 0.00% top
1 root 7 0 428K 428K wait 0:00 0.00% 0.00% init
281 root 7 0 476K 288K (amp) 0:00 0.00% 0.00% sds
704 root 7 0 1224K 272K select 0:00 0.00% 0.00% intensivo
114 root 7 0 280K 272K (amp) 0:00 0.00% 0.00% deuman
10 root 7 0 272K 272K (amp) 0:00 0.00% 0.00% pci
302 root 7 0 520K 260K pause 0:00 0.00% 0.00% cron

```

```

load averages: 1.26, 0.64, 0.21 up 0:01:49:01 00:40:30
47 processes: 3 runnable, 44 sleeping
CPU states: 90.3% user, 0.0% nice, 0.0% system, 1.7% kernel, 0.0% idle
Memory: 2047M Total, 2011M Free, 1996M Contig, 16M Cached

PID USERNAME PRI NICE SIZE RES STATE TIME WCPU CPU COMMAND
702 root 15 0 1224K 260K RUN 2:10 87.79% 87.74% cpu_bound
703 root 11 0 1240K 352K RUN 0:10 10.16% 10.16% mixto
0 root 0 0 3264K 3264K kernel 0:00 0.00% 0.00% (kernel)
33 service 7 0 204K 204K (amp) 0:02 0.00% 0.00% at_wini
256 root 7 0 160K 160K (amp) 0:01 0.00% 0.00% lance
10 root 7 0 432K 432K RUN 0:00 0.00% 0.00% mb
265 service 7 0 1304K 1304K (amp) 0:00 0.00% 0.00% luip
36 root 7 0 1168K 1168K (amp) 0:00 0.00% 0.00% ls
555 root 7 0 2232K 840K select 0:00 0.00% 0.00% syslogd
475 root 7 0 944K 640K select 0:00 0.00% 0.00% dhcpd
650 root 7 0 824K 572K wait 0:00 0.00% 0.00% sh
41 root 7 0 564K 560K (amp) 0:00 0.00% 0.00% procs
705 root 7 0 736K 508K sysctl 0:00 0.00% 0.00% top
1 root 7 0 428K 428K wait 0:00 0.00% 0.00% init
281 root 7 0 476K 288K (amp) 0:00 0.00% 0.00% sds
704 root 7 0 1224K 272K select 0:00 0.00% 0.00% intensivo
114 root 7 0 280K 272K (amp) 0:00 0.00% 0.00% deuman
10 root 7 0 272K 272K (amp) 0:00 0.00% 0.00% pci
302 root 7 0 520K 260K pause 0:00 0.00% 0.00% cron

```

```

load averages: 1.25, 0.90, 0.30 up 0:01:50:05 00:41:40
47 processes: 2 runnable, 45 sleeping
CPU states: 100% user, 0.0% nice, 0.0% system, 0.0% kernel, 0.0% idle
Memory: 2047M Total, 2011M Free, 1996M Contig, 16M Cached

PID USERNAME PRI NICE SIZE RES STATE TIME WCPU CPU COMMAND
702 root 9 0 1224K 260K RUN 3:04 77.90% 77.90% cpu_bound
703 root 7 0 1240K 352K select 0:23 16.65% 16.65% mixto
0 root 0 0 3264K 3264K kernel 0:00 1.17% 1.17% (kernel)
33 service 7 0 204K 204K (amp) 0:02 0.00% 0.00% at_wini
256 root 7 0 160K 160K (amp) 0:01 0.00% 0.00% lance
10 root 7 0 432K 432K RUN 0:00 0.00% 0.00% mb
265 service 7 0 1304K 1304K (amp) 0:00 0.00% 0.00% luip
36 root 7 0 1168K 1168K (amp) 0:00 0.00% 0.00% ls
555 root 7 0 2232K 840K select 0:00 0.00% 0.00% syslogd
475 root 7 0 944K 640K select 0:00 0.00% 0.00% dhcpd
650 root 7 0 824K 572K wait 0:00 0.00% 0.00% sh
41 root 7 0 564K 560K (amp) 0:00 0.00% 0.00% procs
705 root 7 0 736K 508K sysctl 0:00 0.00% 0.00% top
1 root 7 0 428K 428K wait 0:00 0.00% 0.00% init
281 root 7 0 476K 288K (amp) 0:00 0.00% 0.00% sds
704 root 7 0 1224K 272K select 0:00 0.00% 0.00% intensivo
114 root 7 0 280K 272K (amp) 0:00 0.00% 0.00% deuman
10 root 7 0 272K 272K (amp) 0:00 0.00% 0.00% pci
302 root 7 0 520K 260K pause 0:00 0.00% 0.00% cron

```

3. Resultados globales y discusión

Tras la culminación de las fases de desarrollo, se confirma que el proyecto THE MATCOM MINIX ha sido completado satisfactoriamente, cumpliendo con el 100 % de los criterios de aceptación establecidos en la orientación original. El estado final de los componentes es el siguiente:

- **Instalación y Personalización:** El entorno Virtual Box/MINIX es plenamente operativo y el archivo `/etc/motd` refleja correctamente la identidad de la facultad.
- **Depuración de pthread:** Se resolvió el fallo en `pthread_mutex_trylock`, permitiendo que el programa de prueba supere todas las validaciones (PASS) sin quedar bloqueado en llamadas sucesivas.
- **Comando tree:** La herramienta recorre de forma recursiva cualquier ruta proporcionada, gestionando la profundidad mediante indentación visual y evitando ciclos infinitos mediante el uso de `lstat`.
- **Modificación del Scheduler:** Se implementó con éxito la política de penalización en el servicio `sched`, logrando que el sistema identifique y reduzca la prioridad de procesos CPU-bound de forma dinámica.

Análisis y Lecciones Aprendidas

La discusión técnica de este proyecto gira en torno a la transición de un sistema de planificación estático a uno dinámico. Una de las lecciones más importantes sobre el funcionamiento interno de MINIX es que, por defecto, el sistema tiende a tratar a los procesos de usuario con un esquema de prioridades similares o estáticas dentro de sus colas. Al intervenir el planificador, se comprendió que mantener una equidad absoluta (ejecutar procesos con igual prioridad) puede ser ineficiente cuando coexisten tareas interactivas y tareas intensivas en cómputo. La implementación de la penalización por agotamiento de quantum permitió observar cómo el sistema operativo puede "aprender" del comportamiento del proceso en ventanas temporales para favorecer la interactividad, un principio fundamental de los algoritmos MLFQ estudiados en clase.

En cuanto a la concurrencia, la depuración de la biblioteca de hilos evidenció que incluso errores mínimos en la gestión del estado de un mutex pueden derivar en condiciones de interbloqueo (*deadlock*) que inhabilitan la ejecución de un programa. Por último, el desarrollo del comando `tree` reforzó la importancia del uso de llamadas al sistema POSIX (`opendir`, `readdir`, `lstat`) para la manipulación directa del sistema de archivos. El uso de `lstat` resultó crítico: se aprendió que la abstracción de archivos en UNIX/MINIX incluye punteros (enlaces simbólicos) que, de no ser manejados correctamente mediante la obtención del estado del enlace y no del destino, pueden comprometer la estabilidad del algoritmo recursivo. En conclusión, los resultados obtenidos son plenamente coherentes con los objetivos de aprendizaje práctico, demostrando una integración efectiva entre la teoría de sistemas operativos y la implementación real.

4. Conclusiones

Tras la ejecución de este proyecto, se concluye que los objetivos generales han sido cumplidos satisfactoriamente, logrando consolidar el aprendizaje teórico mediante la intervención directa en el núcleo y las bibliotecas de MINIX. La práctica permitió una

comprensión profunda de cuatro pilares fundamentales de los sistemas operativos: la personalización del entorno, la sincronización de hilos, la navegación del sistema de archivos y la planificación de procesos.

En cuanto a los componentes específicos, se alcanzó un 100 % de éxito en todas las actividades:

- La abstracción de hilos y concurrencia fue comprendida al diagnosticar y corregir el error de recursión infinita en `pthread_mutex_trylock`, lo que reforzó la importancia de diferenciar entre capas de compatibilidad y funciones nativas para evitar estados de interbloqueo o fallos críticos de memoria.
- La gestión del sistema de archivos se aplicó mediante el comando `tree`, donde el uso de llamadas al sistema como `opendir`, `readdir` y, especialmente, `lstat`, demostró ser vital para el manejo seguro de la recursión y la prevención de ciclos infinitos causados por enlaces simbólicos.
- La planificación de ejecuciones se abordó mediante la transición de un esquema de prioridades estáticas a una política dinámica de penalización en el scheduler. Se aplicaron con éxito conceptos de algoritmos tipo MLFQ, permitiendo que el sistema "aprenda" del comportamiento de los procesos para favorecer la interactividad frente a tareas intensivas en CPU.

En última instancia, esta experiencia ha sido fundamental para la formación en Ciencia de la Computación, ya que permitió cerrar la brecha entre los conceptos abstractos discutidos en las conferencias y la implementación técnica real en un sistema operativo funcional. La capacidad de diagnosticar fallos a nivel de kernel y proponer soluciones rigurosas constituye una competencia técnica invaluable para el desarrollo de software de sistemas.

5. Referencias consultadas

- MINIX 3 Project. (s.f.). *The MINIX 3 Operating System*. Recuperado de <http://www.minix3.org/>.
- Tanenbaum, A. S. (2014). *Modern Operating Systems* (4ta ed.). Pearson.
- Tanenbaum, A. S., & Woodhull, A. S. (2006). *Operating Systems: Design and Implementation* (3ra ed.). Prentice Hall

6. Declaración de uso de IA

Partes del proyecto donde se usó IA:

- Redacción técnica del informe: Estructuración de las secciones y refinamiento del lenguaje para cumplir con el tono formal exigido por la facultad.
- Búsqueda de información: Localización de datos específicos dentro de la documentación oficial de MINIX y los libros de texto de la asignatura.

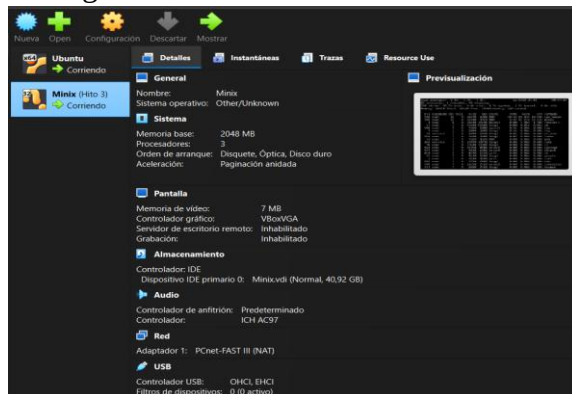
Naturaleza del uso:

- Asistencia en la redacción: Se utilizó la herramienta para organizar las ideas y borradores iniciales en párrafos coherentes y técnicamente rigurosos, siguiendo la estructura de la plantilla oficial.

- Investigación: Agilización en la consulta de referencias bibliográficas (como Tanenbaum) y especificaciones del estándar POSIX para asegurar la validez de la implementación técnica.

7. Anexos

- [KamAlien/minix: Official MINIX sources - Automatically replicated from gerrit.minix3.org](https://kamalien.com/minix)
- Configuración de Minix:



- Comando tree

```
minix > commands > tree > C: tree.c > tree(char*, int)
5 void tree(char *dir, int level)
6 {
7     DIR *directorio = opendir(dir);
8     if(directorio == NULL)
9     {
10        perror("Error al abrir el directorio");
11        return;
12    }
13    struct dirent *hs;
14    struct stat info;
15    while ((hs = readdir(directorio)) != NULL)
16    {
17        if (hs->d_name[0] == '.' && hs->d_name[1] == '\0' || (hs->d_name[0] == '.' && hs->d_name[1] == '.' && hs->d_name[2] == '\0'))
18        {
19            continue;
20        }
21        for(int i = 0; i < level+1; i++)
22        {
23            printf(" ");
24        }
25        printf("%s\n", hs->d_name);
26        char path[1024];
27        snprintf(path, sizeof(path), "%s/%s", dir, hs->d_name);
28        lstat(path, &info);
29        if(S_ISDIR(info.st_mode))
30        {
31            int count = level + 1;
32            tree(path, count);
33        }
34        // un if si no es una carpeta solo imprimimos su nombre
35        // caso contrario guardamos la direccion de la carpeta la
36        // convertimos a char y hacemos el llamado recursivo
37        closedir(directorio);
38    }
39    int main(int argc, char *argv[])
40    {
41        // Esto es para el directorio actual
42        if(argc == 1)
43        {
44            tree(".", 0);
45        }
46        // Esto es para un directorio especifico
47        else
48        {
49            // ...
50        }
51    }
52 }
```

```
minix > commands > tree > Makefile
1 PROG = tree
2 SRCS = tree.c
3 MAN=
4
5 .include <bsd.prog.mk>
```

```
C: schedule.c Makefile C: Mixto.c C: CPU_bound.c C: sched
minix > commands > Makefile
1 # Makefile for commands.
2
3 .include <bsd.own.mk>
4
5 SUBDIR= at backup \
6 cawf cdprobe \
7 cleanmp \
8 compress crc cron crontab \
9 DESCRIBE devmand devsize \
10 dhrystone \
11 eject \
12 fix format fsck.mfs \
13 gcov-pull \
14 ifder \
15 intr isoread \
16 loadkeys loadramdisk look lp \
17 lpd lspeil mail MAKEDEV \
18 minix-service mount nt netconf \
19 prep printroot \
20 profile progressbar \
21 ramdisk rawspeed readclock \
22 remsync \
23 rotate setup \
24 slip spell sprofalyze sprofdiff srcrc \
25 svrctl swifi synctree sysenv \
26 term termcap tget \
27 truncate umount \
28 update version vol \
29 writelofs fetch \
30 zmodem pkgin cd pkgin_all pkgin_sets \
31 worldstone updateboot update_asr update_bootcfg \
32 atnormalize dosread fdisk loadfont \
33 autopart part partition playwave \
34 tree \
35 recwave repartition screendump
36
37 .include <bsd.subdir.mk>
```